Summary

- **Goal:** Build a generative model to generate cohesive classical piano music during inference while allowing user to control generation by inputting desired attributes.
- **Architecture ($\approx 49.5M$ Params):** Conditional Variational Auto-Encoder
  - Encoder ($\approx 19.6M$ Params): Embed music into low-dimensional latent space.
  - Latent Space: Low-Dimensional latent space with learned latent features.
  - Conductor ($\approx 1.88M$ Params): Auto-regressively generates condition embeddings for each musical bar conditioned on the latent representation.
  - Decoder ($\approx 28M$ Params): Auto-regressively generates music conditioned on the condition embeddings.
  - Attribute Embeddings: Musical features which also condition decoder.
- **Comparison:** Compare generation quality to a baseline decoder-only transformer.
  - Negative Log-Likelihood on held-out test-set.
  - TODO: Find relevant metrics for comparison.
    - Need one for determining quality of attribute embeddings and that they actually control the conditioned model.
    - Good organization of global latent space Z.
    - Cohesiveness of the music compared to normal transformer.
    - Generation quality.

Data

- **Datasets:** 500 hours high-quality classical piano
  - MAESTRO - 200 Hours (filtered)
  - GiantMIDI-Piano - curated 300 Hours filtered to $4/4$ time signatures and preferred composers: Beethoven, Tchaikovsky, Chopin, Liszt, Schubert, Ravel, Debussy, Rachmaninoff + keywords “waltz”, “sonata”, “nocturne”, “theme”, “fantasia”, “etude”)
- **Pre-Processing**
  - **Train/Val/Test Splits:** $80/10/10$
    - Train: full VAE architecture
    - Val: ELBO for model selection/tuning
    - Test: NLL for unbiased comparison (baseline vs. VAE)
  - **Time Signatures:** Dataset is strictly filtered to 4/4 (including equivalent 2/4 and 2/2) time signatures. This enforces a uniform bar structure (exactly 16 `Position` slots per bar), simplifies the GRU conductor's latent space, and guarantees predictable sequence lengths. `use_time_signatures=False`.

- **Chunking**: Fixed 24-bar chunks after stripping leading silence in initial bars $\tilde{X} = \mathbf{split24bars}(\mathbf{strip_leading_silence}(X)) \approx 600\text{–}1,100$ tokens (avg ≈ 840).
- **Block Size:** A max `block_size` of 1024.
  - Songs $>$ `block_size` will be discarded to ensure model learns from data which ends on a `<Bar>` token.
  - Songs $<$ `block_size` will be padded with `<PAD>`
- **Bar Token:** `<Bar>` token acts as a hard musical boundary. Combined with in-attention, the decoder switches to the correct bar-level condition vector z_k at every bar transition. TRANSFORMER MAGIC!
- **Controllable Attributes:** Compute raw values for Polyphony Rate, Rhythmic Intensity, Velocity Dynamics, and Note Density in each 24-bar chunk. Global quantiles are then calculated across the full training set to map each feature into 8 standardized discrete levels (0–7). These pre-computed integer labels are saved with each chunk. At training time the DataLoader simply loads the binned indices, enabling zero-CPU-overhead lookup of learned attribute embeddings.

- **Kaggle Feasibility:** AI Advice + Personal Experience.
  - **Hardware**: Single T4 (16 GB VRAM) — use `fp16` mixed precision + `torch.cuda.amp.GradScaler()` + PyTorch native attention, batch size 8**–**12 (fits comfortably, no DDP needed).
  - **Training timeline**: ≈ 45–60 min/epoch; plan for 40**–**50 **epochs** total (≈ 30–45 GPU hours) across 2 sessions (within weekly 30-hour free quota).
  - **Workflow**: Save model + optimizer + epoch every 5 epochs to /kaggle/working/ for easy resume (mandatory for stability).

Tokens

REMI (Revamped MIDI): Utilized the `miditok` library to encode music using the REMI representation. Unlike older methods that treat time as a fluid stream of milliseconds, REMI organizes music exactly like sheet music (explicit measures and beats). This provides hard `<Bar>` boundaries, which our hierarchical architecture relies on to swap the latent condition vectors (Z_k) at the start of every new measure.
Vocab: Musical Grammar consisting of 163 tokens.

Token	Example	Meaning	Vocab Size
`<Bar>`	`Bar`	Boundary token between bars and triggers decoder to switch bar-level latent z_k.	1

Token	Example	Meaning	Vocab Size
`[Position]`	`Position_1`, `Position_16`	The note slot within the current bar	16 (Fixed constant for $4/4$)
`[Pitch]`	`Pitch_60`	Specific Piano Key Pressed	88 (Restricted to physical piano keys)
`[Velocity]`	`Velocity_16`	Loudness/dynamics	32 (Quantized bins)
`[Duration]`	`Duration_4.0`	Note length represented in musical beats. Explicitly includes folded sustain pedal resonance up to a maximum of 8 beats (2 measures).	24 (Fixed constant based on `beat_res`)
`<PAD>`	`<PAD>`	Utilized to fill up chunks $<$ `block_size`	1
`<BOS>`	`<BOS>`	Indicates beginning of sequence	1

Example Sequence:

```
<BOS>
<Bar>
Position_0   Pitch_36 Velocity_28 Duration_4.0   // Deep bass C (Loud, rings
for whole bar)
             Pitch_60 Velocity_20 Duration_4.0   // Middle C (Medium loud,
rings)
             Pitch_63 Velocity_20 Duration_4.0   // E-flat (Medium loud,
rings)
             Pitch_67 Velocity_20 Duration_4.0   // G (Medium loud, rings)
Position_12  Pitch_72 Velocity_10 Duration_0.5   // High C melody note
(Soft, short)
<Bar>        // ... sequence continues for Bar 2
```

- **Beat 1 (The Downbeat):** The pianist plays a heavy, emotional C-Minor chord (4 notes simultaneously) and holds the sustain pedal down so it rings out for all 4 beats.
- **Beat 2 & 3:** The pianist's hands lift up. The chord continues to echo, but no new keys are pressed.
- **Beat 4:** A single, quiet, high melody note is played quickly (lasting half a beat).
- **Next Measure:** The next bar begins.
  Tokenizer Config (`miditok.REMI`):

```
config = TokenizerConfig(
    pitch_range=(21, 108),
    beat_res={(0, 4): 4, (4, 8): 2},
    num_velocities=32,
    use_time_signature=False,
    use_sustain_pedals=True,
    use_tempos=False
)
```

- `pitch_range = (21, 108)` : Restricts vocab to the 88 keys of a piano.
- `beat_res = {(0, 4): 4, (4, 8): 2}:` Fine timing for short notes inside a bar, coarser timing for long sustained notes (up to two bars). Keeps Duration vocabulary small and fixed at exactly 24 tokens.
- `num_velocities = 32` : Quantizes raw MIDI volume into 32 discrete bins.
- `use_time_signature = False` : Ignore `[Time-Sig]` tokens, assuming $4/4$ only.
- `use_sustain_pedals = True` : Folds pedal events into `[Duration]` tokens.
- `use_tempos = False` : Reduce temporal modelling complexity for Conductor.

Attribute Embeddings

- **Attributes:** Each can be inferred from the raw MIDI notes of a song.
  - Polyphony Rate: Allows controlling texture/chordal thickness (the average number of simultaneous notes playing at any given position).
  - Rhythmic Intensity: Allows controlling vibe (Calm vs. Intense).
  - Velocity Dynamics: Allows controlling volume (Loud vs. Soft).
  - Note Density: Allows controlling speed/activity (the total number of note onsets per measure).
- **Embedding Mechanism:** Each attribute is first binned into one of 8 discrete levels (0–7) using global quantiles from the training set. These integer levels are passed through learned embedding lookup tables (`nn.Embedding`). Each attribute produces a 32-dimensional vector; the four vectors are concatenated to form a single 128-dimensional attribute condition vector A_k for each bar.
- **Integration:** A_k (128-dimensional) is concatenated with the bar-level latent vector z_k (384-dimensional) to produce the final 512**-dimensional** condition vector $C_k = [z_k;\ A_k]$. This C_k vector is injected into the decoder via in-attention at every layer and timestep. Maintaining this $3:1$ dimensional ratio physically forces the Decoder to prioritize the musical contour (z_k) over the physical attributes (A_k), preventing latent drowning.
- **Purpose:** These embeddings act as explicit, orthogonal control knobs (mixed-effect terms). By providing them directly, the VAE is relieved of the burden of encoding low-level rhythm/dynamics into the global latent z_p, allowing z_p to focus purely on high-level harmonic

and melodic identity. This enables intuitive user control at inference time while preserving musical coherence.

Encoder

- **Encoder:** Takes input chunk x and outputs parameters μ and σ^2, parameters of Gaussian Distribution, approximating the true posterior distribution $p(z_p|x)$ using a network $q_\phi(z_p|x) = \mathcal{N}(z_p; \mu, \sigma^2\mathbf{I})$.
- **Purpose:** Compresses full musical context into a single global latent vector z_p by looking forward and backward (bidirectional transformer), allowing encoder to figure out "global music identity" (overall mood, harmonic contour, melodic shape, structural feel).
- **Reparameterization Trick:** $z_p = \mu + \sigma \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ and z_p is a deterministic transformation of fixed noise source ϵ which enables the gradient flow through μ and σ during backpropagation.
  - If you sample $z_p \sim \mathcal{N}(\mu, \sigma^2\mathbf{I})$ directly, the sampling operation is stochastic and non-differentiable. Gradients cannot flow through the sampling step, so $\frac{\partial \mathcal{L}}{\partial \mu}$ and $\frac{\partial \mathcal{L}}{\partial \sigma}$ become 0 (or undefined). The encoder weights never receive a useful gradient from the reconstruction term in the ELBO.
- **Architecture:** Bidirectional Transformer f_ϕ (BERT-style) taking input chunk x and outputting $f_\phi(x) = (\mu, \log \sigma^2) \in \mathbb{R}^{128} \times \mathbb{R}^{128}$.
  - **Hyper-Parameters:**
    - `n_layers` : 6
    - `n_embed` : 512
    - `n_head` : 8
    - `feed-forward` : $4 \times$ `n_embed` $= 2048$
  - **Parameters (≈ 19.6 Million Total):**
    - **Token Embedding (`nn.Embedding(163, 512)`):** $163 \text{ tokens} \times 512 \text{ dims} = \mathbf{83,456}$ parameters.
    - **Positional Encoding (`nn.Embedding(1024, 512)`):** $1024 \text{ positions} \times 512 \text{ dims} = \mathbf{524,288}$ parameters.
    - **`[CLS]` Token (`nn.Parameter`):** A single learnable vector of size $512 = \mathbf{512}$ parameters.
    - **6 Transformer Layers (`d_model=512, n_head=8, d_ff=2048`):** Each layer contains:
      - **Self-Attention:** 4 linear layers ($Q, K, V, \text{and Output}$). Each is $(512 \times 512 \text{ weights}) + 512 \text{ biases} = 262,656. \times 4 = 1,050,624 \text{ parameters}.$
      - **Feed-Forward:** - Up-projection $(512 \to 2048)$: $(512 \times 2048) + 2048 = 1,050,624 \text{ parameters}.$

        - Down-projection $(2048 \to 512)$:
          $(2048 \times 512) + 512 = 1,049,088$ parameters.
    - **Layer Normalization:** 2 norms per layer (pre-attention, pre-ffn). Each has $512 \text{ scale}(\gamma) + 512 \text{ shift}(\beta) = 1,024$. $\times 2 = 2,048$ parameters.
    - **Total per layer:** $3,152,384$ parameters.
    - **Total for 6 layers:** $6 \times 3,152,384 = \mathbf{18,914,304}$ parameters.
  - **Latent Projections (The Bottleneck):**
    - μ **head (** `nn.Linear(512, 128)` **):** $(512 \times 128) + 128 = \mathbf{65,664}$ parameters.
    - $\log \sigma^2$ **head (** `nn.Linear(512, 128)` **):**
      $(512 \times 128) + 128 = \mathbf{65,664}$ parameters.
- **Initial Embedding & Positional Encoding:** Input tokens are first passed through a learned embedding table `nn.Embedding(vocab_size, n_embed)` to project discrete tokens into continuous space. Self-attention is permutation-invariant, we add a Learned Absolute Positional Encoding `nn.Embedding(block_size, n_embed)` to these token embeddings. While Relative Positional Encoding is popular in modern LLMs, it is computationally unnecessary here; our fixed 1024 `block_size` pairs perfectly with Absolute encoding, and the REMI vocabulary (`[Position_X]`) already explicitly handles internal musical timing.
- **Bidirectional Self-Attention:** No causal mask unlike decoder. Every token attends to every other token.
- **Sequence Pooling**: A learnable `[CLS]` token is prepended to the input sequence x. The transformer processes the sequence, and the final layer's hidden state at index 0 (the `[CLS]` token) acts as the aggregated global representation. This single dense vector is passed through two parallel linear layers which conduct latent space projection:
  - `nn.Linear(n_embed, 128)` : Outputs the μ vector.
  - `nn.Linear(n_embed, 128)` : Outputs the $\log \sigma^2$ vector. Transform $\log \sigma^2$ to get σ using $f(x) = \exp(0.5 \cdot x)$.

- **Mathematical View:**
  - The bidirectional Transformer processes the full tokenized chunk x (with a learnable `[CLS]` token prepended at the start):

$$h = f_\phi(x) \quad \text{(full sequence of hidden states)}$$

  - The final-layer hidden state at the `[CLS]` position is extracted as the global representation h_{CLS}. This single vector is passed through two parallel linear heads:

$$\mu = \text{Linear}(h_{\text{CLS}}), \quad \log \sigma^2 = \text{Linear}(h_{\text{CLS}})$$

- The approximate posterior is then

$$q_\phi(z_p|x) = \mathcal{N}(z_p; \mu, \mathrm{diag}(\sigma^2))$$

with $\sigma = \exp(0.5 \cdot \log \sigma^2)$.

Conductor

- **Conductor (Deterministic Temporal Unrolling Network)**: The Conductor is a deterministic recurrent network that acts as the temporal Arranger. It translates the static, global latent blueprint (z_p) into a temporal sequence of 24 measure-specific condition vectors (z_k).
- **Purpose:** The Conductor bridges the compact global representation z_p with the detailed, temporally-aware guidance needed by the decoder. It is trained purely through the reconstruction loss (no KL term is applied to z_k).
- **Architecture:** 2-layer Gated Recurrent Unit (GRU) with hidden size 256.
  - **Hyper-Parameters:**
    - `num_layers = 2`
    - `hidden_size = 384`
  - **Parameters (≈ 1.88 Million Total):**
    - **Initialization Projection (`nn.Linear(128, 768)`):** (128×768) weights $+ 768$ biases $= \mathbf{99,072}$ parameters.
    - **Temporal Clock (`nn.Embedding(24, 384)`):** $24 \times 384 = \mathbf{9,216}$ parameters.
    - **2-Layer GRU (`input_size=384, hidden_size=384`):** PyTorch GRUs utilize 3 gates (Reset, Update, New). The formula per layer is $3 \times (\text{hidden} \times \text{input} + \text{hidden} \times \text{hidden} + 2 \times \text{hidden})$.
      - **Layer 1:** $3 \times (384^2 + 384^2 + 2 \times 384) = \mathbf{887,040}$ parameters.
      - **Layer 2:** $3 \times (384^2 + 384^2 + 2 \times 384) = \mathbf{887,040}$ parameters.
  - **Input & Initialization:** The 128-dimensional global latent z_p is projected to 768 dimensions via `nn.Linear(128, 768)` and then reshaped into two 384-dimensional hidden states (one per GRU layer) to properly initialize the 2-layer GRU.
  - **Temporal Clock:** A learned bar embedding (`nn.Embedding(24, 384)`) is added at each step to explicitly tell the GRU its current position in the 24-bar sequence.
  - **Output:** The GRU auto-regressively produces 24 vectors z_k (each 384-dimensional). The entire sequence is deterministic given z_p.
- **Mathematical View:**
  - The 128-dimensional global latent z_p is first projected and reshaped to initialize the two hidden states of the 2-layer GRU:

$$h_0 = \mathrm{Reshape}(\mathrm{Linear}(z_p))$$

with shape $(2, \text{batch}, 384)$, where $h_0^{(1)}$ and $h_0^{(2)}$ are the initial hidden states for layer 1 and layer 2 respectively.

- For each bar $k = 1 \ldots 24$:

$$h_k^{(1)} = \text{GRU}^{(1)}(\text{BarEmbed}_k, h_{k-1}^{(1)})$$

$$h_k^{(2)} = \text{GRU}^{(2)}(h_k^{(1)}, h_{k-1}^{(2)})$$

$$z_k = h_k^{(2)}$$

Hierarchical Latent Space ($z_p \to z_k$)

- **Hierarchical Design:** The architecture deliberately separates a highly compressed global bottleneck from an expanded bar-level condition. This compact-to-expanded pipeline is the key to maintaining long-range musical coherence across all 24 bars while still giving the decoder rich, bar-by-bar generation instructions. This design ensures z_p remains a compact, semantically rich representation while z_k provides the detailed, temporally-aware guidance the decoder needs.
- **Global Latent z_p (128-dimensional):** This is the true, continuous VAE bottleneck. The bi-directional encoder compresses the entire 24-bar chunk into this single vector.
  - **What it Encodes:** It strips away token-level specificities (exact notes and timing) and strictly encodes the high-level "global musical identity" of the whole piece: overall mood, stylistic character, harmonic language, melodic contour, and structural feel.
  - **Smoothness & Interpolation:** Because z_p is regularized by the KL Divergence term of the ELBO loss to approximate a standard normal distribution $\mathcal{N}(0, \mathbf{I})$, the latent space is a completely continuous manifold. This allows for arithmetic operations; sampling a coordinate exactly halfway between two distinct z_p vectors will decode into a coherent, novel sequence that musically interpolates the stylistic qualities of the two original sequences.
- **Bar-Level Latent z_k (384-dimensional):** This is a deterministic sequence of vectors produced by the Conductor network given the global z_p.
  - **What it Encodes:** z_k translates the abstract global style into localized, bar-specific instructions.
  - Dimensional Expansion: z_p undergoes a linear projection (`nn.Linear(128, 768)`) and is then reshaped into two 384-dimensional initial hidden states (one per GRU layer) before being fed into the Conductor GRU. The 128-dimensional global bottleneck forces the VAE to learn foundational musical structure, while the expanded 384-dimensional z_k sequence mathematically matches the Decoder's hidden dimension (`n_embed`) for clean in-attention conditioning.

Decoder

- **Decoder:** Causal (autoregressive) Transformer that generates the next REMI token at each step, conditioned on the full history of previously generated tokens and the bar-level condition vectors c_k.
- **Purpose:** The decoder is the generative core of the model. It takes the global style z_p and bar-level guidance z_k and A_k and auto-regressively produces a coherent sequence of REMI tokens that form a complete musical piece.
- **Architecture:** 8-layer Causal Transformer Decoder (GPT-style) utilizing an In-Attention conditioning mechanism.
  - **Hyper-Parameters:**
    - `n_layers` : 8
    - `n_embed` : 512
    - `n_head:` 8
    - `feed-forward` : $4 \times$ `n_embed` $= 2048$
    - `block_size:` 1024
  - **Parameters (≈ 28 Million Total):**
    - **Token Embedding (`nn.Embedding(163, 512)`):** $163 \times 512 = \mathbf{83,456}$ parameters.
    - **Positional Encoding (`nn.Embedding(1024, 512)`):** $1024 \times 512 = \mathbf{524,288}$ parameters.
    - **8 Transformer Layers (`n_embed=512, n_head=8, d_ff=2048`):** Each layer contains:
      - **Masked Self-Attention:** 4 linear layers (Q, K, V, O). Each is $(512 \times 512) + 512 = 262,656$. $\times 4 = 1,050,624$ parameters.
      - **In-Attention Projection (W_{in}):** `nn.Linear(512, 512)` to transform C_k before summation. $(512 \times 512) + 512 = 262,656$ parameters.
      - **Feed-Forward:**
        - Up-projection: $(512 \times 2048) + 2048 = 1,050,624$ parameters.
        - Down-projection: $(2048 \times 512) + 512 = 1,049,088$ parameters.
      - **Layer Normalization:** 2 norms per layer. Each has $512 \text{ scale} + 512 \text{ shift} = 1024$. $\times 2 = 2,048$ parameters.
      - **Total per layer:** $3,415,040$ parameters.
      - **Total for 8 layers:** $8 \times 3,415,040 = \mathbf{27,320,320}$ parameters.
    - **Output Logit Head (`nn.Linear(512, 163)`):** $(512 \times 163) + 163 = \mathbf{83,619}$ parameters.
    - **Condition Vectors (Input):** For each bar k, the 384-dimensional bar latent z_k and 128-dimensional attribute vector A_k are concatenated to form the 512-dimensional

condition vector $C_k = [z_k;\ A_k] \in \mathbb{R}^{512}$.

- **In-Attention Mechanism (Skip-VAE style):** To heavily enforce the condition without the massive parameter overhead of Cross-Attention, the condition vector is injected via In-Attention. At every decoder layer and every timestep, C_k is passed through a learned square projection (W_{in}) and added element-wise to the 512-dimensional hidden state:
  - $\tilde{h}_l^t = h_l^t + \text{Linear}_{W_{\text{In}}}(C_k)$
  - The modified hidden state $\tilde{h}_l^t$ is then passed to self-attention. Applied at **every layer**, this ensures the global and bar-level guidance strictly influences the entire generation process, preventing latent drowning.

- **Output:** At each position, a linear head projects the final hidden state to vocabulary logits (163), followed by softmax to produce the probability distribution over the next REMI token.

- **Mathematical View:**
  - For each timestep t belonging to bar k:

$$h_l^t = \text{TransformerLayer}_l(h_{l-1}^{<t}, C_k)$$

  - The In-Attention injection C_k is added at every layer l. The final hidden state of the last layer L is projected to logits to determine the next token:

$$p_\theta(x_t \mid x_{<t}, z_p, A) = \text{Softmax}(\text{Linear}(h_L^t))$$

Training Objective (The ELBO Loss)

- **Training Paradigm:** The model is trained end-to-end using teacher forcing. The objective is to maximize the Evidence Lower Bound (ELBO) of the data, which mathematically translates to minimizing a composite loss function comprising two distinct terms: Reconstruction Loss and KL Divergence (with Annealing).
- **Loss Formulation:**

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{Recon}} + \beta \cdot \mathcal{L}_{\text{KL}}$$

  - **Reconstruction Loss ($\mathcal{L}_{\text{Recon}}$)**
    - **Purpose:** Ensures the Decoder accurately recreates the original REMI sequence given the latent variables.
    - **Mechanism:** Standard auto-regressive next-token prediction. It is calculated using Categorical Cross-Entropy loss between the Decoder's output logits and the ground-truth sequence X.
    - **Math:**

$$\mathcal{L}_{\text{Recon}} = -\mathbb{E}_{q_\phi(z_p|x)}[\log p_\theta(X \mid z_p, A)]$$

(where the decoder is conditioned on the full hierarchical vector $C_k = [z_k; A_k]$ at every bar via in-attention).

- **Regularization Loss & β-Annealing ($\mathcal{L}_{\text{KL}}$)**
  - **Purpose:** Regularizes the global latent space z_p (128-dimensional) to approximate a standard normal prior $\mathcal{N}(0, I)$, ensuring the space is continuous and smoothly interpolatable.
  - **The Posterior Collapse Threat:** Because the Decoder is a powerful autoregressive Transformer, it is highly susceptible to posterior collapse (ignoring z_p entirely and relying solely on causal history).
- **The β-Annealing Solution:** We use a β-VAE schedule. β controls the weight of the KL term.
  - Epoch 0: $\beta = 0.0$ (reconstruction dominates).
  - Over the first 20 epochs: β is linearly annealed to a target of 0.1 (tunable; common value for music VAEs to avoid collapse while preserving expressivity).
  - **Math:**

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_\phi(z_p|x) \parallel \mathcal{N}(0, I))$$

  (KL applied only to the global z_p; the deterministic conductor has no KL term).

- **Training Hyperparameters (Kaggle T4 Feasibility)**
  - **Optimizer:** AdamW (`lr = 3e-4` , `weight_decay = 0.01`).
  - **Precision:** Mixed Precision (FP16) using `torch.cuda.amp.GradScaler()` to maximize batch size on the 16 GB T4 GPU.
  - **Batch Size:** 8 to 12 (maximum that fits in VRAM without OOM).
  - **Schedule:** Train for ≈40–50 epochs. Expected runtime is 45–60 minutes per epoch, split across two Kaggle weekly quota sessions.
  - **LR Scheduler:** `torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50, eta_min=1e-5)` (decays from `3e-4` to `1e-5` over the full 40–50 epochs).
- **Temporal Alignment: GPU-Optimized Condition Broadcasting:** Since z_k is dynamically sampled during the forward pass, C_k cannot be precomputed offline without breaking the gradient graph. To prevent CPU bottlenecks during training, alignment is handled via **upfront index pre-computation**.
  - During data preprocessing, an integer tensor (`bar_indices`) is created for every sequence, mapping each token to its corresponding measure index (0 to 23).
  - During the forward pass, the Conductor generates the `[batch, 24, 384]` latent sequence. PyTorch advanced indexing (`Z[:, bar_indices]`) is used to instantly broadcast the condition vectors across the 1024-token sequence in a single, highly optimized CUDA operation. This guarantees perfect temporal alignment with zero Python `for`-loop overhead.

- **Offline Data Preprocessing (Raw MIDI $\rightarrow$ .pt files):** Goal is to convert 500 hours of MIDI into zero-math, ready-to-train integer tensors to bypass Kaggle CPU bottlenecks.
  - Parse & TokenizeFilter raw MIDI strictly to $4/4$ time.
    - Tokenize using `miditok` into the 163-token REMI vocabulary.
    - Slice into exact 24-bar chunks.
    - Drop chunks > 1024 tokens; `<PAD>` chunks < 1024 tokens.
  - Extract & Encode Tensors: For every 24-bar chunk, generate four exact integer arrays:
    - X (Input): $[1024]$ integer array of REMI tokens.
    - Y (Target): $[1024]$ integer array (X shifted by 1 for autoregressive next-token prediction).
    - `bar_indices`: $[1024]$ integer array mapping every token in X to its corresponding measure (0–23). Used for instant C_k broadcasting.
    - A (Attributes): Compute the 4 raw metrics per bar (Polyphony, Intensity, Velocity, Density). Globally quantize them into 8 discrete integer bins (0–7). Shape: $[24, 4]$.
  - Save & Serve
    - Batch these 4 arrays and save to disk as flat PyTorch files (e.g., batch_001.pt).
    - Result: During training, the DataLoader does absolutely no math. It simply loads .pt files and instantly passes `[x, y, bar_indices, attributes]` to the GPU.

Inference

- **Music Generation:** The primary goal is to generate a short, coherent classical piano piece (24 bars ≈ 10–30 seconds) from a sampled global latent vector, while giving the user full real-time control over musical attributes.
- **Controllable Attributes (The Phased UI):** Attribute embeddings A_k (128-dimensional) act as explicit, orthogonal control knobs. Each of the four attributes (Polyphony, Intensity, Velocity, Density) is binned into 8 levels (0–7).
  - The UI exposes these controls and supports **1, 2, or 3 phases**. For a 24-bar piece, the user can set Phase 1 (bars 1–8) to "low density", Phase 2 (bars 9–16) to "high polyphony", and Phase 3 (bars 17–24) to a loud crescendo. This creates a natural musical arc without requiring model retraining.
- **Inference Process (Step-by-Step)**
  1. **Prior Sampling:** Sample a global latent vector $z_p \sim \mathcal{N}(0, I)$ from the prior. *(Note: Users can also spherically interpolate between two known z_p vectors).*
  2. **Temporal Unrolling:** Pass z_p through the deterministic 2-layer GRU Conductor to obtain the sequence of 24 bar-level latent vectors $z_{1:24}$.
  3. **Condition Formulation:** For each bar k, construct the condition vector $C_k = [z_k; A_k]$ by concatenating z_k with the user-specified phased attribute vector A_k.

4. **Autoregressive Generation:** Start the Decoder with a `<BOS>` token. At every step, the Decoder processes the token history and the current condition vector C_k via **In-Attention**.
5. **Grammar Masking:** To guarantee musically legal output without post-hoc repair, the tokenizer's vocabulary constraints are enforced before sampling. Logits of invalid tokens are masked to $-\infty$.
   - `<Bar>` must be followed by `[Position]`
   - `[Pitch]` must be followed by `[Velocity]`.
6. **Dynamic Switching:** When the model naturally generates a `<Bar>` token, the Python inference loop catches it, increments the bar counter, and physically switches the injected condition vector to C_{k+1} for the subsequent tokens.
7. **Rendering:** Stop after 24 `<Bar>` tokens or the 1024-token limit. Detokenize the sequence back to a MIDI file using `miditok`, and render to audio using `pretty_midi` and `FluidSynth`.

- **Mathematical View:**
  - A single piece $X = (x_1, \ldots, x_T)$ is generated as:

$$z_p \sim \mathcal{N}(0, I)$$

$$\{z_k\}_{k=1}^{24} = \text{Conductor}(z_p)$$

$$x_t \sim p_\theta(x_t \mid x_{<t}, C_{k(t)}) \quad \text{where } C_k = [z_k; A_k]$$

  - Here, $k(t)$ is the bar index of token t, determined strictly on-the-fly by counting the number of previously generated `<Bar>` tokens. The full joint probability is:

$$p_\theta(X \mid z_p, A) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}, C_{k(t)})$$

- **Important Considerations**
  - **Temperature & Nucleus Sampling:** Sampling is performed token-by-token. A softmax temperature ($\tau \approx 1.0$) and nucleus sampling (Top-$p \approx 0.9$) are used to maintain diversity while preventing gibberish.
  - **Reproducibility:** By seeding the random number generator and saving the sampled z_p and A_k UI settings, any generated piece can be exactly reproduced later.
  - **Deployment & Inference Architecture:** The user interface will be deployed as a modern React web application (built with Vite). The frontend will capture the user's phased attribute settings and send an asynchronous request to a containerized Python backend (FastAPI + Docker). The specific cloud hosting platform for the inference container is currently TBD, but will be selected to optimally handle the model's autoregressive generation payload. Once the backend generates the sequence, it is

detokenized via `miditok` and rendered to audio in the browser using `pretty_midi` and FluidSynth.

Future Work & Extensions

- **Phase 2: Adversarial Latent Disentanglement ($\mathcal{L}_{\text{Adv}}$):** While the base model relies on the 2:1 condition ratio and β-annealing to separate high-level style from low-level attributes, there remains a risk of **latent entanglement**. The Encoder might still implicitly encode low-level dynamics (like note density or velocity) into the global latent z_p, creating conflicting signals when a user adjusts the attribute knobs during inference. To mathematically force z_p and the Attribute Embeddings (A_k) to be orthogonal (disentangled), future iterations will implement a **Gradient Reversal Layer (GRL)**.
  - **Satisfiability Test:** Validate need by using a fixed z_k and testing with varying A_k to see if outputted song style changes.
  - **Mechanism:** A small auxiliary Multi-Layer Perceptron (MLP) Discriminator will be introduced during training. The Discriminator takes z_p as input and attempts to classify the ground-truth attributes A.
  - **The GRL Integration:** We apply a Gradient Reversal Layer between the Encoder and the Discriminator. While the Discriminator minimizes its classification error, the reversed gradients actively penalize the Encoder for leaking attribute data.
  - **Result:** The Encoder is forced to strip all rhythmic and dynamic information out of z_p, leaving it to encode *only* high-level harmonic identity. This guarantees that the user's UI controls (A_k) act as strictly independent generation variables.
- **Guided Generation via Monte Carlo Tree Search (MCTS):** Currently, the autoregressive Decoder relies on Nucleus (Top-p) and Temperature sampling. While effective, standard left-to-right sampling is susceptible to error accumulation (where one bad token derails the entire subsequent measure). To optimize for global musical coherence, inference could be upgraded using a **Monte Carlo Tree Search (MCTS)** decoding strategy.
  - **The Value Function:** We would train an auxiliary "Critic" network (or utilize a pre-trained music quality metric) to act as the value function. This Critic evaluates the musicality, harmonic consistency, and adherence to the condition vector C_k of a partially generated sequence.
  - **The Search Process:** Instead of greedily picking the next token, the Decoder would simulate multiple future rollouts (branches). The MCTS algorithm balances exploration (trying rare tokens) with exploitation (following high-probability paths), guided by the Critic's reward scores.
  - **Result:** This transforms generation from a reactive, step-by-step prediction into a forward-planning search, heavily reducing hallucinated dissonances and ensuring complex musical cadences resolve properly over long time horizons.

- Attribute control validation (project TODO): Compute per-bar stats on generated vs. real (polyphony, density, etc.) and report correlation with input bins. This directly addresses “quality of attribute embeddings” and is easy to implement with miditok/pretty_midi. For latent-space organization, plot t-SNE of zp colored by composer/genre.